Pulse CBD — Individual Report (Draft)

Oct 30, 2026 · @Anthony

Executive Summary

Pulse CBD projects live Sydney public transport onto a physical scale model of the CBD: every bus, train, Metro, light rail and ferry in the model's footprint appears as a moving arrow, coloured by whether it is on time, in traffic, or crowded. My contribution came in two phases: an early prototype pipeline and dashboard (August to October), and then, after the team consolidated on a shared engine, a set of features and data-quality fixes in that engine during the final integration week.

The central outcome of my work is a **stop-level history of lateness and crowding**, built from the live feed. Every time a vehicle passes a stop, the system records how late and how full it was. After two days of recording this held about 59,000 stop visits across roughly 1,500 stops, and it showed a clear pattern: at 366 of the 402 stops with enough data to compare, vehicles arrived later when crowded than when seats were available.

Building this exposed problems in the source data that affected the whole project, not just my feature:

- **Placeholder zeros.** About 7.5% of delay readings were an exact 0 that meant "no live prediction", not "on time". For Sydney Trains it was 68% of readings. These were silently counted as on-time.
- **Feeds that list the whole trip.** Metro, both light rail lines, ferries and NSW TrainLink send every stop of a trip, including ones already passed, so the code read the first stop's delay (always 0) instead of the next stop's.
- **Glitches.** A small number of readings were impossible, such as Inner West light rail trams reported 41.7 hours late.

Fixing these changed what the projection shows: more vehicles now honestly display as "no schedule data" instead of falsely green, and light rail and Metro now show real delays.

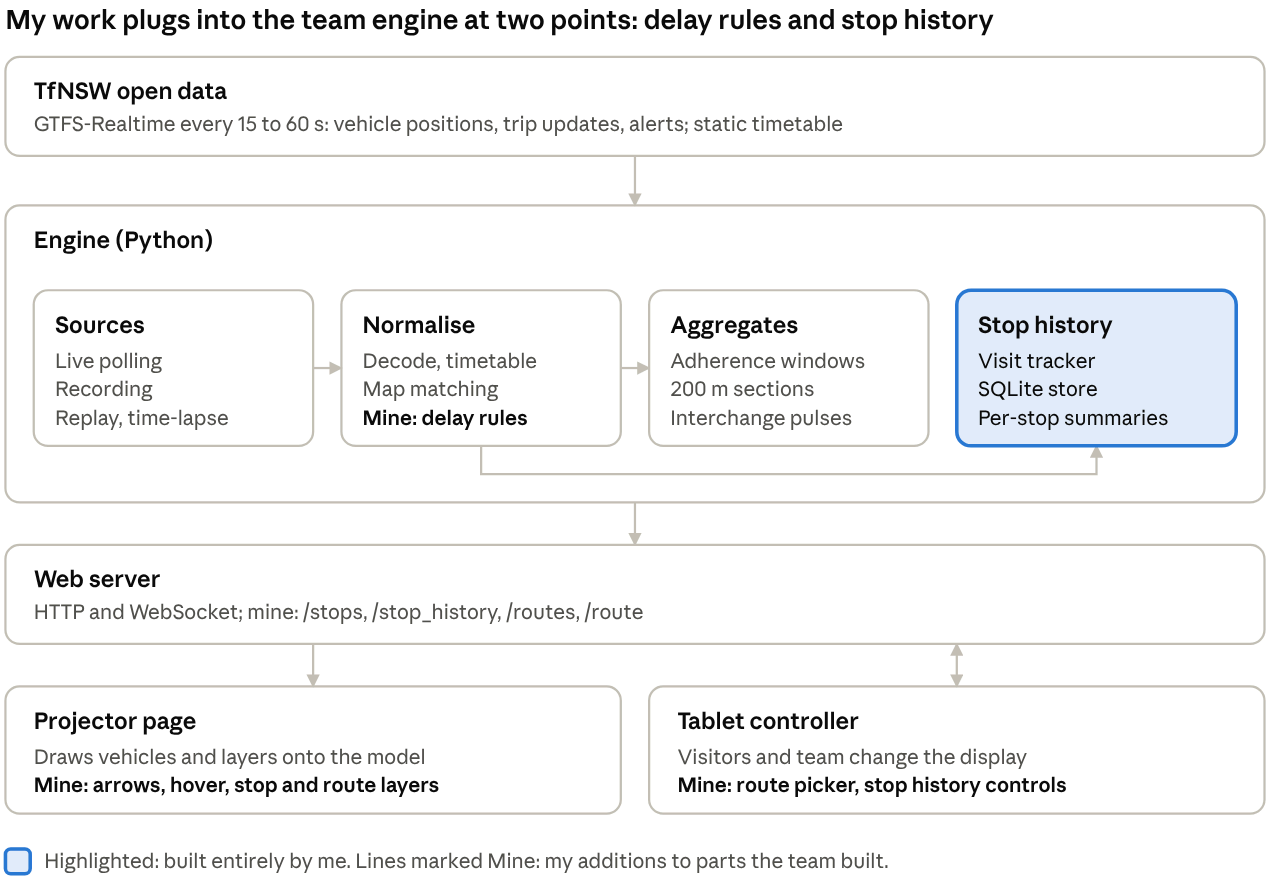
Alongside this I added directional arrows for all modes, exact delay on hover, a stop history layer with median and average views, a route filter that singles out chosen routes and their stops, and the fixes needed to run the server on Windows. Across the final week this came to about 1,670 lines added over 34 files, and the automated test suite grew from 148 to 175 tests, all passing.

The main limitation is the amount of data: stop history covers about two weekdays, almost none of it in the morning peak, so peak-hour results are not yet reliable. Recording continues until the showcase on 6 November.

Project Context

The team's brief was to make the CBD's transport network legible at a glance on a physical model, for a public showcase on 6 November. The system has three parts: a Python engine that polls Transport for NSW's GTFS-Realtime feeds every 15 to 60 seconds, a projector page that draws the network onto the model, and a tablet controller that visitors and the team use to change what is shown.

The engine follows a fixed pipeline. Raw protobuf feeds are decoded, joined to the static timetable, snapped onto route shapes (map matching), and classified by schedule adherence. Rolling aggregates then summarise traffic and crowding along 200 m street sections. A small web server pushes vehicle frames and display settings to the pages over a WebSocket.



System architecture · team engine with my contributions marked

The stop history is the one component I built from scratch; elsewhere my work changes how existing parts behave.

The team agreed a few design rules early, recorded in `docs/DECISIONS.md`, and these shaped my work:

- **No basemap.** The model itself is the map; the projection only adds data.
- **Nothing blended.** Lateness, traffic and crowding stay separate measures, never one combined score (the only exception is the explicitly labelled "pressure" layer).
- **Route sections, not grid squares,** for anything that moves along a route.

How the work was divided. Yoseph Kim built the engine's core: the renderer, projector calibration, interchange pulses, recording and replay, the BOAM and demand layers, and adapting polling to TfNSW's daily request quota. [Add other team members' roles here.] My early work was a separate prototype in `template-repository-main/` (a pipeline and dashboard). When the team consolidated on Yoseph's engine, I moved my effort into that codebase and focused on stop-level history, data quality, and visitor-facing interaction. Section 3 separates these clearly.

My own early design notes ("Anthony notes" in the group notes) set the direction for my final-week work: record occupancy and lateness at each stop, and keep the two separate so they can be compared rather than blended.

Individual Contribution

I made 47 of the repository's 68 commits (as `0rangeGoose`). They fall into two phases: a prototype that the team later replaced, and features and fixes in the team's shared engine during the final week. All final-week work is on the `integration` branch and covered by automated tests.

Phase 1: prototype pipeline and dashboard (28 Aug to 28 Oct)

Before the team settled on a single codebase, I built a separate prototype in `template-repository-main/`: a data pipeline (ingestion, normalisation, analytics, publish) and a web dashboard.

Date	What I built	Commit
18 Sep	Independent bus, train and Metro toggles	`6e626cd`

Date	What I built	Commit
17 Sep	Historical trend comparison by named period, showing the latest day per period rather than a blend	`cc2bfe7`, `0a1ecb6`
17 Sep	Real demand data (TZP24 employment, TZ21 zones) tracked with Git LFS	`01fbc20`
16 Sep	Routes layer: congestion along real streets; layers made independent toggles	`d73a047`, `7e9ba31`
15 Sep	Projector-only map view with keyboard layer switching	`7b65881`
15 Sep	Demand layer: TZP24 employment joined onto the congestion grid	`8c12d85`, `aa774e4`
14 Sep	Bus bunching as a congestion signal; grid shrunk to 150 m	`566520d`
11 Sep	CBD congestion scoring; map clipped to the model's surveyed footprint	`c7919e4`, `5262372`
28 Aug	Initial transit data pipeline; Git LFS fix for static GTFS data	`4ad0414`, `8a6a3e2`

This prototype was superseded when the team adopted Yoseph's engine. Some of its ideas were sound (independent layer toggles, separating vehicle modes, period comparisons) and reappear in the final system. Its grid-based congestion score was not: it conflicted with the team's later rule to use route sections for anything that moves along a route. I discuss what I learned from this in Section 5.

Phase 2: the team engine (29 to 30 Oct)

In the final integration week I worked directly in `engine/` and `renderer/`. Measured against the team's last commit before mine (`ff65c32`), this added about 1,670 lines across 34 files, including six new test files, and the suite grew from 148 to 175 tests.

Date	What I built or fixed	Commit
30 Oct	Route filter: single out up to 8 routes with their stops; fixed colour per route	`7ecca02`

Date	What I built or fixed	Commit
30 Oct	Median and average stop delay; filter for impossible readings	`e220af5`
30 Oct	Placeholder-zero fix, full-trip feed fix, history clean-up	`d3d4a58`
30 Oct	Stop history layer coloured by delay or crowding	`0630169`
30 Oct	Directional arrows for every mode, including Sydney Trains	`16e2217`, `a27b1bb`
29 Oct	Stop-visit recording, stop markers, hover tooltips, Windows support	`7346651` and following

Key decisions

Use the feed's own "next stop", not GPS distance. The old prototype guessed when a bus was "at" a stop by checking whether it was within 25 m. I checked the live feed and found the TripUpdate's `stop_id` was filled for 97.4% of trips, so the engine now detects a stop visit when that value changes. This is exact rather than a guess, and it works for stops the vehicle never pauses at.

Show "unknown" rather than a false "on time". After the placeholder-zero fix, about 70% of Sydney Trains display as grey "no schedule data" at any moment. This makes the projection look less complete, but the alternative was presenting missing data as good news.

Median by default, average as an option. Delay is skewed: most vehicles are slightly late and a few are very late. The median describes the typical bus; the average also counts the bad days. I kept both, with the same colour bands, because the gap between them identifies unreliable stops. Edgecliff Station's median was 34 s late but its average was 3 min 35 s.

Fixed colours per route, assigned by the server. When several routes are singled out, each takes a slot from an 8-colour palette chosen so neighbouring colours stay distinguishable for colour-blind viewers. The server assigns slots, so removing one route never repaints the others, and the projector and every tablet always agree.

Challenges

Running on Windows. The engine was written on macOS. On my Windows laptop it failed to start (`fcntl` does not exist on Windows), a date format crashed (`%-d` is not valid there), and the controller page returned a 500 error. The last one came from reading text files without stating an encoding, so Windows used cp1252 instead of UTF-8. I fixed it at the crash site, then searched the whole codebase and fixed the same pattern in about 15 other places.

A 101 MB file blocking the push. A large GeoJSON file had been committed and then deleted, but it still sat in two unpushed commits, so GitHub rejected the push. I backed up the branch, removed the file from only those unpushed commits, and confirmed the final files were byte-for-byte unchanged before pushing.

Results that looked right but were not. The stop history first showed CBD stops as greener than suburban ones. Rather than accept that, I tested it: CBD stops sit near the start or end of many routes, where delay is lowest, and have more rail visits. The colour measures delay *inherited on arrival*, not delay *caused* at that place. This is now a stated limitation, and it is the reason to build a "delay gained per section" measure next.

Technical Deep Dive: Stop-Level Lateness and Crowding History

The question from my design notes was simple: at a given stop, are crowded vehicles also the late ones? Answering it needed three things the engine did not have: a reliable way to know when a vehicle passes a stop, a delay value that can be trusted, and a summary per stop that is robust to bad data. The second turned out to be the hardest.

1. Detecting a stop visit

GTFS-Realtime has two relevant feeds. VehiclePosition gives location, heading and crowding but never says which stop the vehicle is at. TripUpdate gives, for each trip, a list of upcoming stops with predicted delays. I use the first upcoming stop as the vehicle's "next stop". When it changes from A to B, the vehicle has just passed A.

The tracker (`engine/history/stop_tracker.py`) keeps, per vehicle, the stop it is heading to and the latest delay and crowding seen while heading there. On a change it emits one visit for the previous stop, using the last reading taken before the vehicle left it:

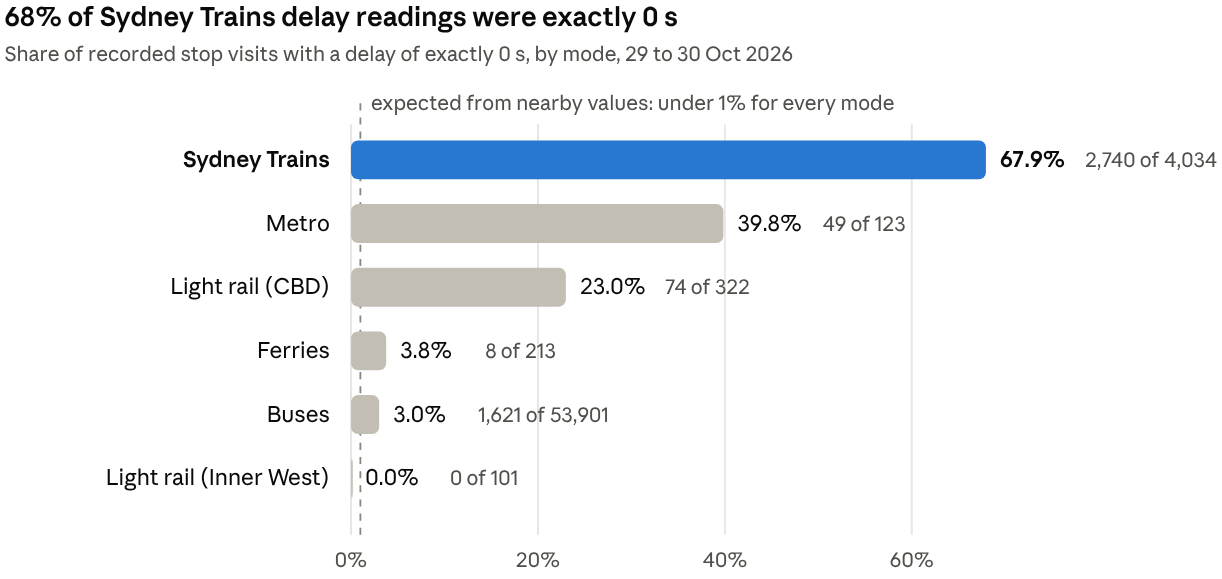
```
prev = self._tracked.get(vehicle.vehicle_key)
if prev is not None and prev.next_stop_id != next_stop_id:
    visit = {"stop_id": prev.next_stop_id, "occupancy_status":
prev.occupancy_status,
             "delay_seconds": prev.delay_seconds, ...}
self._tracked[vehicle.vehicle_key] = _Tracked(next_stop_id=next_stop_id, ...)
```

The tracker does no input or output itself; a separate store (`engine/history/store.py`) writes each visit as one row in a SQLite table. Keeping the logic pure made it easy to test with synthetic vehicles (6 tests), including a missed poll and two vehicles tracked independently.

2. Finding a delay that can be trusted

The first version took the delay from the first stop in the TripUpdate list that had one. Checking the recorded data against what real timing should look like showed this was wrong in three separate ways.

Placeholder zeros. Real delays are spread smoothly around zero, so roughly as many readings should say 0 s as say 3 s or 4 s. Instead, almost every mode reported far more exact zeros than that, as the chart below shows. In 99% of these readings every remaining stop on the trip was also exactly 0, a pattern no real prediction produces, and trips switched in and out of it mid-run. The zeros meant "no live prediction". For buses, TfNSW sometimes says so explicitly with a `NO_DATA` flag, which the code had ignored.



Recorded stop visits, 29 to 30 Oct 2026 · expected = average count at the 20 neighbouring one-second values

Every mode except Inner West light rail reported far more exact zeros than real timing produces; buses look mild only because 1,621 zeros are spread over 53,901 readings.

Feeds that list the whole trip. Buses and Sydney Trains drop stops from the list once served, as GTFS-Realtime convention suggests. Metro, both light rail lines, ferries and NSW TrainLink do not: they send the whole trip from stop 1, every time. The "next stop" was therefore stuck on the trip's origin, whose departure delay is always 0. Every stop visit recorded for those modes was attached to the wrong stop.

Glitches. A few readings were physically impossible: Inner West light rail trams 41.7 hours late, and some buses more than 30 minutes early.

The fix (`engine/normalise/trip_updates.py`) changes how the next stop and its delay are chosen:

```
usable = [(stu, d) for stu in tu.stop_time_update
          if stu.schedule_relationship not in (NO_DATA, SKIPPED)
          and not _already_passed(stu, now)          # predicted time before
the feed's timestamp
          and (d := _stop_delay(stu)) is not None]
if usable:
    next_stop_id = usable[0][0].stop_id or None
    if any(d != 0 for _, d in usable):               # all zeros = no live
prediction
        delay = usable[0][1]
```

The cost is small: by the expected counts, about 125 genuinely on-time readings are now treated as unknown, against about 4,400 placeholders removed. Readings outside 30 minutes early to 3 hours late are dropped as glitches before any statistic; the 3-hour limit keeps the worst real case I found, long-distance trains about 2.6 hours late. I also cleaned the stored history, with a backup: 4,522 stored zeros became unknown, and 762 visits recorded at the wrong stop were deleted.

3. Summarising per stop

Each stop's summary (`stop_summaries`) gives the median and the average delay, the share of vehicles arriving with few seats or fuller, and the median delay split into seated and crowded arrivals. Two thresholds protect against thin data:

- a stop is coloured only with at least 20 visits;
- the seated and crowded split needs at least 5 visits on each side.

Colour bands are fixed in minutes (under 1, 1 to 2, 2 to 4, over 4 for delay) rather than recalculated from the data. If the bands were percentiles, the same share of stops would always show red, and a bad afternoon could never look worse than a good one.

4. Validation and limits

After the fix, no live vehicle reports a delay of exactly 0, and CBD light rail shows a real median of 113 s late instead of a constant 0. Among the 27 tests I added in the final week are cases for each failure above, for example a trip of all zeros, a passed origin stop, and a `NO_DATA` stop followed by a real one.

The method still has known limits, each measured on recorded data:

- **Missed stops.** About 4% of stops are passed between two 15-second polls and get no visit.
- **Backward jumps.** About 2% of next-stop changes go backwards when TfNSW revises a prediction, creating a few false visits.
- **Inherited delay.** A stop's colour shows how late vehicles already are on arrival, not whether that place causes delay.
- **Little peak data.** Only 3 stops have enough morning-peak visits to be coloured so far.

Reflection & Learning

What worked

Checking the data before trusting the feature. The stop history looked finished on the first day: it recorded visits, coloured stops and passed its tests. The problems only appeared when I plotted the distribution of delays and compared it with what real timing should look like. One spike at exactly 0 s led to three separate bugs, one of which affected every vehicle on the projection, not just my layer. The lesson I take from this is that tests check the code does what I intended, not that what I intended matches the data.

Building on the team's conventions. In the engine I reused existing patterns rather than inventing new ones: the offscreen canvas layers, the shared control state with server-side validation, and the generic toggle buttons on the controller. New features slotted in with small changes, and the team's design rules (nothing blended, unknown is not good news) gave clear answers to questions I would otherwise have argued about.

What did not work, and what I would do differently

I built a parallel prototype for too long. From August to October I developed my own pipeline and dashboard instead of contributing to the shared engine. Some ideas carried over, but most of the code did not, and its grid-based congestion score contradicted a rule the team later adopted. I would agree the codebase and core design rules with the team in the first two weeks.

I started recording too late. Stop history needs weeks of data, and I began collecting it six days before the showcase. The morning peak is almost empty (3 stops have enough visits). If I started again, continuous recording would begin as soon as the feed worked.

My commit history is hard to read. Messages such as "a", "stuff" and "Antohny changes" do not tell a teammate what changed, and committing a 101 MB file cost time to undo. I would make smaller commits with descriptive messages and check file sizes before committing.

Peak windows were assumed, not measured. The project's AM and PM peaks (7 to 9 am, 4:30 to 6:30 pm) were the team's choice in code, not an official definition. I would derive them from the BOAM data or our own recordings and state the evidence.

How the project developed my skills

- **Transport data:** how GTFS-Realtime and the static timetable fit together, and how much operators differ in how they fill the same fields.
- **Transport engineering concepts:** official on-time definitions (a Sydney bus counts as on time from 59 s early to 5 min 59 s late), the difference between punctuality and regularity, and why delay *inherited* differs from delay *caused*.
- **Data quality and statistics:** comparing observed against expected counts to find artefacts, medians versus means for skewed data, and outlier fences (quartiles plus or minus 3 times the interquartile range) to separate abnormal from impossible readings.
- **Software engineering:** working in an unfamiliar codebase, testing logic separately from input and output, portable file handling across operating systems, and repairing git history safely.

Use of AI assistance

I used an AI coding assistant (Claude Code) throughout the final week to explore the codebase, write and test code, and investigate the data. I set the direction from my design notes, chose between the options it proposed, and checked results against the live

system and recorded data. [Describe in your own words how you used it, in line with the unit's AI policy.]

[Add a personal reflection: what you found hardest, what you are proudest of, and how working with the team went.]

References

1. Aemmer, Z., Ranjbari, A. and MacKenzie, D. (2022). Measurement and classification of transit delays using GTFS-RT data. *Public Transport*, 14(2). Publication page, PDF
2. TransitCenter (2020). *Scheduling and dispatching frequent bus service*. PDF
3. Transport for NSW. Opal peak and off-peak fares. transportnsw.info
4. Transport for NSW. Buses on-time running. Open data portal
5. Audit Office of NSW. Bus contracts in metropolitan Sydney. audit.nsw.gov.au
6. Greater Sydney Bus Contract on-time definition (59 s early to 5 min 59 s late), as reported by Oz Arab Media. [Verify against the contract's Schedule 4 before submission.]
7. Pulse CBD team repository, `integration` branch; design rules in `docs/DECISIONS.md`.

Appendix A: Where my code lives

Area	Files
Stop-visit detection and storage	`engine/history/stop_tracker.py`, `engine/history/store.py`
Next stop, delay and data-quality rules	`engine/normalise/trip_updates.py`
Heading for trains from the route shape	`engine/normalise/map_match.py` (`heading_at`), `engine/normalise/enrich.py`
Route filter	`engine/publish/route_focus.py`, `engine/publish/control.py` (`routes`, `route_slots`)
Endpoints	`engine/publish/server.py`: `/stops`, `/stop_history`, `/routes`, `/route`
Display and controller	`renderer/index.html`, `renderer/controller.html`

Area	Files
Windows support	`engine/sources/budget.py`, `engine/sources/replay.py`, plus encoding fixes in about 15 files
Tests	`tests/test_stop_tracker.py`, `test_history_store.py`, `test_trip_updates.py`, `test_route_focus.py`, `test_control.py`, `test_enrich.py`, `test_map_match.py`

Appendix B: Figures to add

- [ ] Screenshot of the projection with routes 333, L2 and T1 singled out
- [ ] Screenshot of the stop history layer, median versus average
- [ ] Delay distribution by mode with outlier cut-offs (generated during the analysis)
- [ ] Photo of the projection on the physical model